

DL-판교-9반-P298이재겸+P307정원준

SUMMARY

1. 결측치 처리, 데이터 증강

라벨이 없는 행은 제거하고, Wafer 이미지를 회전과 뒤집기를 하여 소수 클래스 데이터를 증강했습니다.
→ 학습 데이터 품질과 클래스 균형이 개선되어 학습 성능이 향상되었습니다.

2. 규제(Dropout), EarlyStopping

Conv/FC 층에 Dropout을 추가하고 Valid loss 기준 조기 종료를 적용했습니다.
→ Overfitting이 줄어 Test 데이터 지표의 안정성이 확보되었습니다.

3. 입력 Filter 재구성, AvgPooling으로 변경

Conv 필터 구성을 조정하고 MaxPooling을 AvgPooling으로 교체했습니다.
→ LOC, Scratch 등 불량 부위의 밀도 정보가 convolution 학습에 더 잘 반영되었습니다.

4. 데이터 관점 변경 학습 (극좌표계 추가)

직교좌표 이미지에 극좌표(반경·각도) 변환 이미지를 함께 학습(Fusion)했습니다.
→ 다른 관점의 특징을 함께 보며 더 많은 패턴을 학습해 판별력이 향상되었습니다.

결론: 모델이 일반화된 성능을 올리기 위해, Macro-F1, Recall 점수가 고르게 높은 4번모델을 최종선택하였습니다

- 최적 모델의 성능 :

지표	0. Baseline	4. 최종선택 모델	변화	최종선택 모델 (4번)	평가 값
Train-Accuracy	95.78%	87.50%	▼ 8.28%p	Test-Accuracy	87.77%
Valid-Accuracy	84.34%	87.24%	▲ 2.90%p	Test-Macro-F1	82.65%
Valid-Macro-F1	79.34%	83.31%	▲ 3.97%p	Test-Recall (mean)	85.89%
Valid-Recall (소수 클래스)	63.63%	93.30%	▲ 29.67%p	Test-Recall (소수)	95.00%

분석 시나리오

1. 문제 정의

1.1 배경 및 도메인

반도체 웨이퍼 생산 공정에서 각 다이(die)의 양품/불량 여부를 웨이퍼 맵(2D 그리드) 형태로 기록한 **WM-811K** 데이터셋(LSWMD.pkl)을 사용합니다. 웨이퍼 위에 불량이 발생하는 **공간적 패턴**(중심에 몰림, 가장자리에 링 형태, 스크래치 자국 등)은 설비 이상이나 공정 문제의 단서가 되기 때문에, 이 패턴을 자동으로 분류하는 것이 목적입니다.

1.2 문제 유형

이미지(2D) 다중 클래스 분류 문제입니다.

- 입력(X): 웨이퍼 다이의 상태(0=측정 안 됨, 1=정상, 2=불량)를 담은 가변 크기 2D 배열(**waferMap**)
- 출력(y): 9개 클래스 중 하나
 - Center**: 웨이퍼 중심 부근에 결함이 집중
 - Donut**: 웨이퍼 중앙을 제외한 고리 모양 불량
 - Edge-Loc**: 웨이퍼 가장자리의 특정 부분에만 불량
 - Edge-Ring**: 웨이퍼 전체 가장자리를 따라 발생하는 환형 불량
 - Loc**: 웨이퍼의 특정 부위에만 무작위로 발생하는 불량
 - Near-full**: 웨이퍼 대부분을 차지하는 불량
 - Random**: 웨이퍼 전체에 무작위로 퍼져있는 불량
 - Scratch**: 물리적 긁힘이나 손상으로 인한 직선형 불량
 - none**: 정상데이터

1.3 해결해야 할 문제

- Data Class 불균형**: 전체 81만여 장 중 약 78.6만 장(97%)이 **none** (정상)이고, 나머지 8개 불량 유형은 149~9,680장 수준으로 매우 적습니다. (특히 **Near-full** 은 149장).
- Overfitting**: 모델이 과적합 되어있어 early stopping 으로 해결을 고려해 볼 수 있음
- 규제 추가**: Dropout, Weight Decay, BatchNorm 을 사용하여, 모델이 데이터를 외우는 상황을 해소해 볼 수 있음
- 모델 구조 개선**: 층을 더 쌓거나, 필터 수를 조정하여 성능 개선을 가져볼 수 있음

1.4 목표(Objective)

- 불균형하고 크기가 제각각인 Wafer Image에서, CNN으로 9가지 웨이퍼 불량 패턴(정상 포함)을 분류하되, 다수 class에 치우치지 않고 소수 불량 유형까지 잘 잡아내는 모델을 만드는 것
- 데이터를 보고 시나리오를 구상하고, 코드로 반영해보며 모델을 개선시켜 나가는것.
- Accuracy, Recall, F1-macro 지표를 확인해 보며 가장 최선이라고 생각되는 모델을 선정

1.5 평가지표 선택 이유

클래스 불균형 때문에 단순 Accuracy는 **none** 을 잘 맞히기만 해도 높게 나와 지표로 사용하기엔 부적합 할 수 있습니다.

- **Accuracy**
- **Recall(Mean), Macro F1**
- **클래스별(Class-wise) Recall**

따라서 위 지표들을 함께 확인해, 실제로 중요한 소수 불량 유형(Donut, Near-full 등)을 잘 잡아내는지까지 평가합니다.

1.6 성공 기준

- 데이터 전처리부터, Conv 깊이, 필터 수, FC 구조, Dropout 등을 조정해 성능(평가지표에 표시)를 개선

2. 데이터 확인 및 사전 처리

2.1 데이터 확인

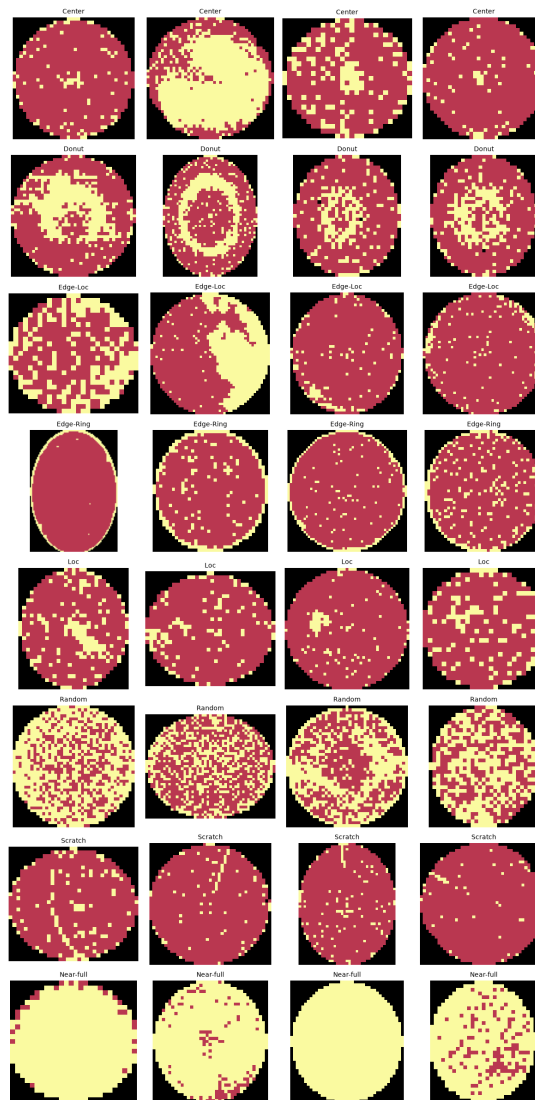
2.1.1 데이터 별로 값 확인

- 각 컬럼별 데이터를 확인하였다.

```
<Class Distribution>
failureType
none          785938
Edge-Ring     9680
Edge-Loc      5189
Center        4294
Loc           3593
Scratch       1193
Random        866
Donut         555
Near-full     149
Name: count, dtype: int64
```

2.1.2 데이터 이미지 직접 확인

- 데이터가 잘 불러와진것을 확인 할 수 있다
- 이미지의 노란색 부분이 불량부분이다.



2.2 데이터 사전 처리

2.2.1 Train / Vailidation / test 로 변환하여 검증단계를 추가

```
X_temp, X_test, y_temp, y_test = train_test_split(
    X, y, test_size=0.15, random_state=42, stratify=y
)
X_train, X_valid, y_train, y_valid = train_test_split(
    X_temp, y_temp, test_size=0.15 / 0.85, random_state=42, stratify=y_temp
)
```

- Train / Test 만으로 분리하던 과정에 valid 데이터셋을 추가
- train : valid : test = 85% : 15% : 15% 로 비율을 설정
- 따라서 model은 valid data로만 성능을 평가하고 학습하며, Test data는 최종 평가에서 한번만 사용이 됨
- random_state=42를 고정하여, 다른 모델에서도 동일한 결과를 나올 수 있도록 함

2.2.2 null과 none을 구분 안하는것을 확인 → 과감하게 제거 필요

- 기존코드

```

# 라벨(failureType) 전처리
# 원본 데이터의 failureType은 [['Center']]처럼 리스트 안에 있거나 비어있음
# 이를 일반 문자열로 변환하고, 라벨이 있는 데이터는 제외하는 과정
df['failureNum'] = df.failureType
df['trainTestNum'] = df.trainTestLabel

mapping_type = {
    'Center': 0, 'Donut': 1, 'Edge-Loc': 2, 'Edge-Ring': 3, 'Loc': 4,
    'Near-full': 5, 'Random': 6, 'Scratch': 7, 'none': 8
}

# 라벨 정보가 있는 데이터만 추출
df = df.drop(['waferIndex', 'dieSize', 'lotName'], axis=1)
df['failureType'] = df['failureType'].apply(lambda x: x[0][0] if len(x) > 0 else 'none')

# 불량 유형별 개수 확인
print("\n<Class Distribution>")
print(df['failureType'].value_counts())

```

- 기존 코드는 failureType에 값이 null 일 때, 값을 none(정상)으로 바꿈
- 실제로 638507 개의 데이터가 라벨링이 안되어있었는데, 정상 데이터로 판별됨
- 결측 데이터 삭제 후 각 컬럼 데이터 양

라벨 없는 데이터 638,507개 삭제 (811,457개 -> 172,950개)

```

<Class Distribution>
failureType
none      147431
Edge-Ring  9680
Edge-Loc   5189
Center     4294
Loc        3593
Scratch    1193
Random     866
Donut      555
Near-full  149
Name: count, dtype: int64

```

2.2.3 데이터 불균형 해소

- 데이터 불균형이 심하게 있었음

```

<Class Distribution>
failureType
none      147431
Edge-Ring  9680
Edge-Loc   5189
Center     4294
Loc        3593
Scratch    1193
Random     866
Donut      555
Near-full  149
Name: count, dtype: int64

```

- 데이터의 불균형이 존재.
- 같은 불량 데이터중에서도 Edge-Ring 데이터는 9680개인 반면, Near-full 불량 데이터는 149개
- 데이터 증강 아이디어
 - 데이터가 원형 모양이고 상하좌우 구분이 없어보였음.
 - 0, 90, 180, 270도 돌리기를 각각 하고 각 케이스에서 뒤집기를 하여 데이터를 증강
 - 따라서 최대 8배만큼 데이터를 증강시킬 수 있음
 - 또한 모든 데이터를 똑같이 증강시키면 불균형 문제가 전혀 해소되지 않기에, 최대 데이터 수를 정한 후 부족한 데이터들을 증강시켰다.
- 데이터 증강 과정
 - 주요 코드 일부

```

ARGET_COUNT = 4000

def rotate_flip_variants(img):
    """(1,H,W) 웨이퍼맵의 원본을 제외한 7가지 대칭 변형을 반환"""
    return [
        np.rot90(img, 1, axes=(1, 2)),
        np.rot90(img, 2, axes=(1, 2)),
        np.rot90(img, 3, axes=(1, 2)),
        np.flip(img, axis=1),
        np.flip(img, axis=2),
        np.flip(np.rot90(img, 1, axes=(1, 2)), axis=1),
        np.flip(np.rot90(img, 1, axes=(1, 2)), axis=2),
    ]

```

- train data 를 먼저 split 한 후 train 데이터에 한하여, 데이터를 증강 시켰다.

```
=== 클래스 불균형 완화 (Train set만 회전/뒤집기 증강) ===
class 0: 3006개 -> 4000개 (+994)
class 1: 389개 -> 3112개 (+2723)
class 2: 3632개 -> 4000개 (+368)
class 3: 6776개 (이미 목표 이상, 증강 생략)
class 4: 2515개 -> 4000개 (+1485)
class 5: 104개 -> 832개 (+728)
class 6: 606개 -> 4000개 (+3394)
class 7: 835개 -> 4000개 (+3165)
```

- class 5 (Near-full) 데이터는 아직 비교적 부족하지만, 이전보다는 불균형이 해소 되었기에 우선 진행시켜보기로 하였다.

2.2.4 입력 데이터 크기 RESIZE

- 입력데이터의 크기가 제각각이라 일정한 size로 맞춤
- 데이터의 size가 바뀌면서 채워야하는 부분, 또는 합쳐지는 부분은 근처의 값을 이용하여 보간법을 이용하여 채운다.
- code

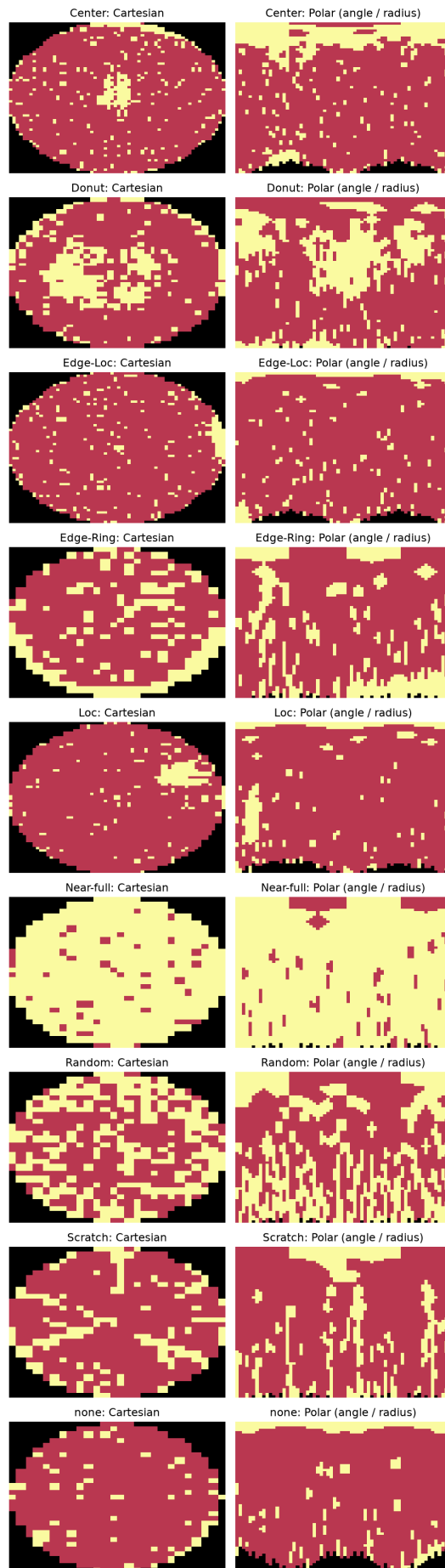
```
# CNN 입력 처리 위한 전처리 (Resize)
# 웨이퍼크기가 제각각이라 고정된 크기 (64*64)로 맞춤

def preprocess_wafer_map(wafer_map, target_size=(64, 64)):
    """
    CNN 입력용으로 이미지 크기를 통일하는 함수
    OpenCV의 resize 등을 사용해도 무방
    """
    # 웨이퍼 맵을 target_size로 변환 (Nearest Neighbor 보간법 권장: 0,1,2 값을 유지하기 위해)
    # 다만, skimage resize는 기본적으로 float로 반환하므로 주의 필요
    resized_map = resize(wafer_map, target_size, anti_aliasing=False, preserve_range=True, order=0)

    # 0, 1, 2 정수값 유지
    return resized_map.astype(int)
```

2.2.5 이미지 데이터의 모양을 보다가 새로운 아이디어를 떠올림. (극좌표 전환)

- 데이터의 모양이 원형 데이터인것을 확인
- 이 pixel들을 극좌표로 변형하여 그 데이터에 cnn을 적용시켜보는 아이디어를 떠올림
- 아래 이미지에서 우측 데이터가 극좌표로 새롭게 변형된 데이터.
- 새로운 데이터에 Cnn을 적용해보자는 아이디어가 떠오름



3. 분석 및 모델링

3.1 BASE MODEL (기준)

- 교수님의 기존 파일에서 데이터셋을 Train\Valid\Test로 다시 분할
- 결과

#	실험 개요	주요 하이퍼파라미터	Train-Accuracy	Valid-Accuracy	Valid-Macro-F1	Valid-Recall (소수)	비고
0	Baseline	-	95.78%	84.34%	79.34%	63.63%	

3.2 데이터 전처리만 적용 (none 삭제 + 부족한 데이터 증강)

3.2.1 아이디어 및 전략

- 데이터를 확인해 보니 결측치처리와 데이터 증강과정이 필요하다는 생각이 들었다.
- 목차 2.2 데이터 전처리에서 상세한 내용을 이미 설명하였고, 그 전처리 결과를 보는 과정이다.

3.2.2 구체적인 적용 방안 (아키텍처)

3.2.3 결과

#	실험 개요	주요 하이퍼파라미터	Train-Accuracy	Valid-Accuracy	Valid-Macro-F1	Valid-Recall (소수)	비고
0	Baseline	-	95.78%	84.34%	79.34%	63.63%	
1	+ 결측치처리 + 데이터증강	TARGET_COUNT = 4000	98.10%	83.68%	79.20%	93.73%	

3.3 Overfitting 해소 (Dropout, Early Stopping 적용)

3.3.1 아이디어 및 전략



- Train Loss 는 하락하지만, Valid Loss는 더 줄지않아 과적합을 의심해 보고 있다.
- 규제 (Dropout)과 Early Stopping을 적용해보았다.

3.3.2 구체적인 적용 방안 (아키텍처)

- Early Stop

```

# ---- Early Stopping ----
# valid loss가 개선되면 best_model_state 갱신, 아니면 patience_counter 증가
if val_epoch_loss < best_valid_loss:
    best_valid_loss = val_epoch_loss
    best_model_state = copy.deepcopy(model.state_dict())
    patience_counter = 0
else:
    patience_counter += 1
    if patience_counter >= patience:
        print(
            f"Early stopping at epoch {epoch + 1} "
            f"(best valid loss: {best_valid_loss:.4f})"
        )
        break

# 가장 valid loss가 낮았던 시험의 가중치로 복원
if best_model_state is not None:
    model.load_state_dict(best_model_state)

```

- o patiecnr 변수 값을 3으로 정하여, 3번연속 validation loss 개선이 없으면 학습 멈춤
- o 기존의 가장 낮았던 loss로 돌아간다.
- Dropout

```

self.dropout_conv = nn.Dropout(0.25)

# Flatten 후 크기 계산: 64(채널) * 14(가로) * 14(세로) = 12544
self.fc = nn.Sequential([
    nn.Flatten(),
    nn.Linear(64 * 14 * 14, 64),
    nn.ReLU(),
    nn.Dropout(dropout_p), # FC 레이어 Dropout
    nn.Linear(64, 9) # 9개 클래스
])

```

- o Dropout을 0.25만큼 랜덤하게 제거하여 과적합 개선을 시도하였다.

3.3.3 결과

#	실험 개요	주요 하이퍼파라미터	Train-Accuracy	Valid-Accuracy	Valid-Macro-F1	Valid-Recall (소수)	비고
0	Baseline	-	95.78%	84.34%	79.34%	63.63%	
1	+ 결측치처리 + 데이터증강	TARGET_COUNT = 4000	98.10%	83.68%	79.20%	93.73%	
2	+ Dropout + Earlystop	Dropout(0.25) patience = 3	77.91%	84.38%	77.53%	94.7%	

3.4 Filter size 및 pooling 파라미터 수정

3.4.1 아이디어 및 전략

- CNN이 "이 영역에 불량일 얼마나 밀집해 있는가"라는 정보를 스스로 학습해야 하는데, 초반 conv 이를 안정적으로 포착 해야함 필요했고, 이후 Pooling 방식에 따라 이 밀집도(비율) 정보가 왜곡될 수 있다.
- 1. **Filter(필터) 기법** — 학습 가능한 필터에 전적으로 의존하지 않고, **고정된 커널을 제시하고** 5x5, 11x11 크기의 지역 불량률(밀도)을 미리 계산해 별도 채널로 추가한다. 즉 모델이 배워야 할 것을 일부 사전 계산해서 명시적으로 제공한다.
- 2. **Pooling(풀링) 기법** — 밀도 채널은 "0/1 존재 여부"가 아니라 "비율" 값을 담고 있는데, MaxPool은 국소 최댓값만 남기므로 이 비율 정보를 왜곡할 수 있다. AvgPool로 바꾸면 비율 정보를 더 잘 보존할 것이라는 가설을 세웠다.

3.4.2 구체적인 적용 방안 (아키텍처)

- 밀도 채널 계산 (Filter)

```

valid = (x > 0).float() # 유효 다이 영역
failed = (x == 2).float() # 불량 다이
count = F.conv2d(failed, ones_kernel_k, padding=k//2)
n = F.conv2d(valid, ones_kernel_k, padding=k//2)
density_k = count / n.clamp_min(1) * valid # 배경은 0으로 유지 (분모 왜곡 방지)

```

- o k=5, k=11 두 크기로 각각 계산 → 원본 1채널 + 밀도 2채널 = **총 3채널** 입력으로 확장
- o 첫 conv 레이어 `in_channels` 를 1→3으로 변경 (파라미터 576개 증가에 그림, 구조 자체는 거의 유지)

- Pooling 교체

- o 구조는 완전히 동일하게 두고 `MaxPool2d` ↔ `AvgPool2d` 만 바꿔 두 변형을 각각 학습 (`density_max` , `density_avg`)

3.4.3 결과

#	실험 개요	주요 하이퍼파라미터	Train-Accuracy	Valid-Accuracy	Valid-Macro-F1	Valid-Recall (소수)	비고
0	Baseline	-	95.78%	84.34%	79.34%	63.63%	
1	+ 결측치처리 + 데이터증강	TARGET_COUNT = 4000	98.10%	83.68%	79.20%	93.73%	
2	+ Dropout + Earlystop	Dropout(0.25) patience = 3	77.91%	84.38%	77.53%	94.7%	
3	+ 입력 filter추가 + avgpooling 변경	입력filter (전략부분에서 설명) AvgPool2d(kernel_size=2, stride=2)	83.11%	86.68%	81.59%	95.7%	

3.5 극 좌표계 변환

3.5.1 아이디어 및 전략

- Wafer 불량 유형이 **중심으로부터의 거리(반지름)와 각도**에 따라 규칙적으로 나타난다. 그러나 기존 CNN은 원본 x, y 격자(Cartesian) 좌표계 이미지만 입력받기 때문에, 이런 원형 패턴을 인식하려면 컨볼루션 필터가 회전 방향마다 별도로 학습돼야 하는 비효율이 있다.
- 동일한 웨이퍼맵을 중심 기준 거리·각도로 재구성한 **극좌표(Polar) 표현**을 추가로 만들고, 원본(Cartesian) 표현과 함께 모델에 입력하여 두 관점을 동시에 학습시킨다.
- 데이터의 상세한 모형은 2.2.5에서 설명하였다.

3.5.2 적용 방안

- Cartesian과 Polar는 좌표계 자체가 다르므로, 픽셀 단계에서 두 이미지를 단순 결합(Early Fusion)하면 같은 위치의 픽셀이 서로 대응되지 않아 의미가 없음
- 따라서 각 표현을 **동일 구조의 CNN 인코더로 독립적으로 학습**시켜 의미 있는 특징 벡터를 추출한 뒤, 그 특징을 결합(concat)하는 방식을 채택
- 두 인코더는 구조는 같지만 가중치는 서로 다르게 학습되어, 하나는 "위치 기반 패턴"을, 다른 하나는 "방사형/각도 기반 패턴"을 전문적으로 학습하도록 유도

3.5.3 모델 구조

- 기존 **BaselineCNN**의 conv 파트를 **WaferEncoder**로 분리·재사용
- Cartesian 전용 **encoder_cart**, Polar 전용 **encoder_polar** 두 개를 독립적으로 생성
- 각 인코더가 64차원 특징 벡터를 추출 → **torch.cat**으로 128차원으로 결합 → **Dropout** → **Linear(128, 9)**로 최종 9개 클래스 분류

3.5.4 결과

#	실험 개요	주요 하이퍼파라미터	Train-Accuracy	Valid-Accuracy	Valid-Macro-F1	Valid-Recall (소수)	비고
0	Baseline	-	95.78%	84.34%	79.34%	63.63%	
1	+ 결측치처리 + 데이터증강	TARGET_COUNT = 4000	98.10%	83.68%	79.20%	93.73%	
2	+ Dropout + Earlystop	Dropout(0.25) patience = 3	77.91%	84.38%	77.53%	94.7%	
3	+ 입력 filter추가 + avgpooling 변경	입력filter (전략부분에서 설명) AvgPool2d(kernel_size=2, stride=2)	83.11%	86.68%	81.59%	95.7%	
4	+ 극좌표 변환 데이터 추가	(위에서 설명)	87.50%	87.24%	83.31%	93.3%	

4. 종합 결과

- 최적화
 - 사용한 전략
 1. 결측치 처리, 데이터 증강
 2. 규제 (Dropout), EarlyStopping
 3. 입력 Filter 재구성, AvgPooling 으로변경
 4. 데이터의 관점을 변경하여 학습 (극좌표계로 변경)
 - 최적화 효과 :
 - **결측치 처리, 데이터 증강** : 학습데이터 전처리, 학습 성능 향상

- **규제, EarlyStopping** : Overfitting 감소로 test data 지표 안정성 보장
- **입력 Filter, AvgPooling 변경**: LOC, Scratch 등은 불량부분의 밀도를 convolution 학습에 반영
- **데이터 변형(극좌표) 추가**: 다른 관점에서 추가 학습후 함께 보아 더 많은 특징을 학습

◦ 최적 성능 :

항목	내용/값
실험 #	#4
적용 기법 조합	Baseline + 결측치처리 + 데이터증강 + Dropout + Earlystop + 입력 filter추가 + avgpooling 변경 + 극 좌표 변환 데이터 추가
Test-Accuracy	87.77%
Test-Macro-F1	82.65%
Test-Recall (mean)	85.89%
Test-Recall (소수)	95.00%

• 실험전략:

#	실험 개요	주요 하이퍼파라미터	Train-Accuracy	Valid-Accuracy	Valid-Macro-F1	Valid-Recall (소수)	비고
1	Baseline	-	95.78%	84.34%	79.34%	63.63%	
2	+ 결측치처리 + 데이터증강	TARGET_COUNT = 4000	98.10%	83.68%	79.20%	93.73%	
3	+ Dropout + Earlystop	Dropout(0.25) patience = 3	77.91%	84.38%	77.53%	94.7%	
4	+ 입력 filter추가 + avgpooling 변경	입력filter (전략부분에서 설명) AvgPool2d(kernel_size=2, stride=2)	83.11%	86.68%	81.59%	95.7%	
5	+ 극좌표 변환 데이터 추가	(전략부분에서 설명)	87.50%	87.24%	83.31%	93.3%	

• 클래스별 Recall 상세 :

클래스	건수	Baseline	실험#1	실험#2	실험#3	실험#4	비고
Center	644	92.9%	91.3%	95.2%	94.9%	95.8%	
Donut	83	73.5%	97.6%	97.6%	97.6%	98.8%	
Edge-Loc	779	82.4%	81.9%	78.3%	88.2%	84.2%	
Edge-Ring	1452	95.2%	97.9%	98.1%	97.0%	97.9%	
Loc	539	75.0%	62.7%	56.8%	62.0%	66.2%	
Near-full	22	81.8%	91.3%	95.7%	95.7%	91.3%	
Random	130	86.2%	92.3%	90.8%	93.8%	96.9%	
Scratch	179	34.1%	64.8%	43.6%	60.9%	54.2%	
none	750	77.6%	67.9%	81.6%	79.2%	89.2%	결측치 처리

결론: 모델이 일반화된 성능을 올리기 위해, Macro-F1, Recall 점수가 고르게 높은 4번모델을 최종선택하였습니다

5. 회고

- 이번 학습을 통해 데이터를 살펴보면 모델의 아키텍처를 만들어 가는 과정의 중요성을 배울 수 있었다.
- 또한 역으로 좋은 성과를 위해 데이터를 증강하는 등 데이터전처리의 중요성 또한 배울 수 있었다.
- Optimizer, Activation 등도 추가로 고려해보고 싶었지만, 전체적으로 우리가 목표했던 평가지표가 많이 상승하였고, 중요한 소수 Class들의 recall값 또한 많이 상승하였으므로 이번 과정에선 여기까지 진행하기로 하였다.
- 이번 보고서에서 다루지 못한 다른 파라미터들은 추후 과정으로 진행해보면 좋을것 같다.
- 또한 정답 데이터 (none label) 은 모든 픽셀이 정답 색(우리 코드에선 빨강) 이라 rule-base로 처리하고 반영해 보아도 좋을것 같다는 생각이 들었다.